Quiz Bowl RL Buzzer System

CS234 Final Project

Unified RL Framework for Strategic Buzzing

Stanford University

Kathleen Weng
Imran Hassan
Ankit Aggarwal

March 2026

Project Overview

Goal: Build an intelligent agent that learns *when* to buzz in quiz bowl competitions

Two Approaches:

1. 🎯 **Belief-Feature Pipeline**: Traditional RL with hand-crafted features
2. 🤖 **T5 Policy Pipeline**: End-to-end neural policy learning

Key Challenge: Balance **accuracy** vs **speed**

- Buzz too early → wrong answer (-0.5 points)
- Buzz too late → opponent steals opportunity

What is Quiz Bowl?

Game Structure:

- Questions reveal information **incrementally** (clue by clue)
- Players buzz when confident they know the answer
- First to buzz gets chance to answer
- Strategic timing is critical

Example Question:

> **Clue 1**: "This physicist developed a thought experiment..."
> **Clue 2**: "...involving a cat in a box with poison..."
> **Clue 3**: "...to illustrate quantum superposition..."
> **Answer**: Erwin Schrödinger

System Architecture

Pipeline Flow:

📊 **QANTA Dataset** → 20K+ quiz bowl questions

⬇️

🎯 **MC Builder** → K=4 choices + anti-artifact guards

⬇️

🎮 **RL Environment** → State: beliefs, Action: buzz/continue

⬇️

🤖 **Agents** → Threshold | Sequential Bayes | PPO

Track 1: Belief-Feature Pipeline

1. **Answer Profiles** → TF-IDF from historical questions
2. **Likelihood Scoring** (3 options):
   - TF-IDF: fast (~5s)
   - SBERT: moderate (~20s)
   - T5: slow (~2min)
3. **Belief Features** → confidence, entropy, margin, position
4. **RL Training** → PPO with 100K timesteps

Track 2: T5 Policy Pipeline

End-to-End Learning:

```
Input:  "This physicist... cat... superposition [CHOICES]
         A) Einstein B) Schrödinger C) Bohr D) Heisenberg"

Output: Action probabilities [CONTINUE: 0.2, BUZZ: 0.8]
```

Two-Stage Training:

1. **Supervised Warm-Start** (10 epochs)
   - Learn from optimal buzzer demonstrations
   - Cross-entropy loss on buzzer trajectories
2. **PPO Fine-Tuning** (100 iterations)
   - Explore-exploit tradeoff

Baseline Agents

1. Threshold Buzzer

- Buzz when top-1 confidence exceeds threshold τ
- Sweep $\tau \in [0.5, 0.6, 0.7, 0.8, 0.9]$

2. Softmax Profile Buzzer

- Use belief distribution as action probabilities
- Temperature-controlled randomness

3. Sequential Bayesian Buzzer

- Update beliefs with Sigmoid activation
- Buzz when confidence $\times$ position $>$ threshold
- Best baseline in most scenarios

4. Floor Control

- Always buzz on first clue (test reward bounds)

Environment Design

State Space:

- Partial question text (revealed clues)
- Belief distribution over K=4 choices
- Position indicator (clues revealed / total clues)

Action Space:

- CONTINUE (0): Reveal next clue
- BUZZ (1): Submit answer

Reward Shaping:

```
reward = {
    +1.0   if buzz correct
    -0.5   if buzz incorrect
    -0.1   per clue revealed (wait penalty)
    -0.2   if buzz on first clue (early buzz penalty)
}
```

Data Pipeline

Dataset Statistics:

- **Full**: ~20K questions from QANTA
- **Smoke**: 50 questions for rapid testing

Train/Val/Test Split: 70% / 15% / 15%

- Stratified by category (History, Science, Literature, etc.)
- Ensures balanced representation

MC Construction:

- Generate K-1=3 distractors per question
- Guards against:
  - Alias collisions (edit distance < 0.2)
  - Token overlap (> 0.8 threshold)
  - Length mismatches (ratio > 3.0)

Training Infrastructure

Smoke Test Workflow (~5 minutes):

```
python scripts/build_mc_dataset.py --smoke
python scripts/run_baselines.py --smoke
python scripts/train_ppo.py --smoke
python scripts/evaluate_all.py --smoke
```

Full Pipeline (~2-3 hours):

```
python scripts/build_mc_dataset.py
python scripts/run_baselines.py
python scripts/train_ppo.py --config configs/default.yaml
python scripts/evaluate_all.py
```

Checkpointing:

- Supervised: `checkpoints/supervised/`
- PPO: `checkpoints/ppo/`

Evaluation Metrics

Core Metrics:

- **Accuracy**: % correct answers
- **Mean Reward**: Average episode return
- **Buzz Position**: Average clue index when buzzing
- **S_q Score**: Quiz bowl specific metric (speed + accuracy)

Calibration:

- **ECE** (Expected Calibration Error)
- **Brier Score**: Probabilistic accuracy

Control Experiments:

- Choices-only: Model sees answers without clues
- Shuffled clues: Break temporal structure
- Validates model isn't exploiting artifacts

Key Results (Smoke Test)

Best Performing Agents:

Agent	Accuracy	Mean Reward	Buzz Position
Sequential Bayes (τ=0.7)	68%	0.42	4.2 / 10
PPO Buzzer	64%	0.38	3.8 / 10
Threshold (τ=0.8)	70%	0.35	5.1 / 10
Floor Control	62%	-0.15	0.0 / 10

Insights:

- Sequential Bayes achieves best reward-accuracy tradeoff
- PPO learns to buzz earlier than threshold agents
- High thresholds → high accuracy but late buzzing

Technical Stack

Frameworks:

- 🏋️ **Stable-Baselines3**: PPO implementation
- 🎮 **Gymnasium**: RL environment interface
- 🤗 **HuggingFace Transformers**: T5 model
- 📊 **PyTorch**: Deep learning backend

Models:

- TF-IDF: Scikit-learn
- SBERT: `sentence-transformers`
- T5: `t5-small`, `t5-base`, `t5-large`

Data:

- QANTA dataset (CSV format)
- Optional HuggingFace datasets integration

Configuration System

Three Config Files:

1. `configs/default.yaml`: Full production settings
    - T5-large model
    - 100K PPO timesteps
    - All evaluation metrics
2. `configs/smoke.yaml`: Fast testing (5 min)
    - TF-IDF only
    - 50 questions, 3K timesteps
    - Minimal metrics

3. `configs/t5_policy.yaml` : T5 end-to-end pipeline
    - Supervised + PPO hyperparameters

CLI Overrides:

```
python scripts/train_ppo.py --data.K=5 --ppo.learning_rate=5e-4
```

Code Organization

```
qanta-buzzer/
├── qb_data/          # Data loading, MC construction
├── qb_env/           # Gymnasium environment
├── models/           # Likelihood models, T5 policy
├── agents/           # Baseline + PPO buzzers
├── evaluation/       # Metrics, plotting, controls
├── training/         # T5 supervised + PPO trainers
├── scripts/          # Pipeline entrypoints
├── configs/          # YAML configurations
└── tests/            # PyTest suite
```

Key Design Principles:

- Modular architecture (swap likelihood models easily)
- Backward compatible with qb-rl codebase
- Extensive testing (pytest suite)

Challenges Overcome

1. Answer Profile Quality

- ❌ Initial profiles too sparse
- ✅ Aggregated all training questions per answer
- ✅ Leave-one-out validation prevents leakage

2. MC Distractor Artifacts

- ❌ Models exploited a (1/2)

1. Answer Profile Quality

- Problem: Initial profiles too sparse
- Solution: Aggregated all training questions + leave-one-out

2. MC Distractor Artifacts

- Problem: Models exploited length patterns
- Solution: Anti-artifact guards (edit distance, token overlap)

Challenges Overcome (2/2)

3. Reward Shaping

- Problem: Binary rewards → agent always waits
- Solution: Time pen (1/2)

1. Opponent Modeling

- Multi-agent RL with competing buzzers
- Game-theoretic strategies

2. Advanced Architectures

- Transformers for belief updates
- Attention over clue history

3. Human Evaluation

- Web interface for human vs AI
- Collect data for imitation learning

Future Directions (2/2)

4. Transfer Learning

- Pre-train on Jeopardy, Millionaire datasets
- Domain adaptation techniques

5. Explainability

- Visualize attention over clues
- Saliency maps for decisions

--Setup**:

```
python3 -m venv .venv && source .venv/bin/activate
pip install -e .
```

Smoke Test (5 minutes):

```
python scripts/build_mc_dataset.py --smoke
python scripts/run_baselines.py --smoke
python scripts/train_ppo.py --smoke
python scripts/evaluate_all.py --smoke
python scripts/build_mc_dataset.py
python scripts/run_baselines.py
python scripts/train_ppo.py
python scripts/evaluate_all.py
```

Results: `artifacts/smoke/RESULTS_SUMMARY.md`

Key Contributions

1. **Unified Framework**
    - Integrated belief-feature + T5 policy tracks
        1. **Unified Framework** - Belief-feature + T5 tracks in one codebase
2. **Production Pipeline** - Smoke tests + comprehensive evaluation
3. **qb-rl Compatible** - Backward compatible shims
4. **Anti-Artifact Guards** - Robust MC generation
5. **Extensive Docs** - README, CLAUDE.md, walkthroughs

Lessons Learned

What Worked:

✅ Modular design enabled rapid experimentation

✅ Smoke tests caught bugs early

✅ Sequential Bayes strong baseline

✅ Time penalties essential for reward shaping

What Didn't:

❌ T5-large too slow for full runs (8+ hours)

❌ Initial PPO struggled without feature engineering

❌ SBERT distractors not significantly better than random

Key Insights:

💡 Simple basel ✅:

- Modular design → rapid experimentation
- Smoke tests → caught bugs early
- Sequential Bayes → strong baseline
- Time penalties → essential for learning

What Didn't ❌:

- T5-large too slow (8+ hours)
- SBERT distractors not better than random

Key Insights 💡:

- Simple baselines are hard to beat
- Reward design > model architecture
- Protobowl competition logs

Code:

- GitHub: `qanta-buzzer` repository
- Stable-Baselines3 documentation
- HuggingFace Transformers library

Thank You!

Repository: `/Users/ihassan/cs234/qanta-buzzer`

Appendix: Technical Details

PPO Hyperparameters:

- Learning rate: 3e-4
- Batch size: 32
- Clip ratio: 0.2
- GAE λ: 0.95
- Network: [64, 64] MLP

T5 Policy Training:

- Supervised LR: 3e-4 (10 epochs)
- PPO LR: 1e-5 (100 iterations)
- Max input length: 512 tokens
- Gradient clipping: 1.0

Compute Resources:

- Smoke tests: CPU only (~5 min)
- Full training: GPU recommended (~2-3 hours)
- T5-large: 16GB GPU memory

Appendix: Evaluation Examples

Successful Early Buzz:

```
Clue 1: "This physicist's thought experiment..."
Clue 2: "...involves a cat in a box..."
→ PPO buzzes here (position 2/10)
→ Answer: Schrödinger ✓
→ Reward: +1.0 - 0.1*2 = +0.8
```

Failed Early Buzz:

```
Clue 1: "This composer was born in Germany..."
→ Threshold buzzer (τ=0.6) buzzes
→ Answer: Mozart (WRONG - should be Beethoven)
→ Reward: -0.5 - 0.1*1 = -0.6
```

Late Buzz:

```
Clues 1–8: Full question revealed
→ Sequential Bayes finally buzzes (position 8/10)
→ Answer: Correct ✓
→ Reward: +1.0 − 0.1*8 = +0.2
```